

Guía Docente — Sesión 4: Redes, Volúmenes y Persistencia

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026) **Docente:** Ing. Cristian Jampier Chileno Segundo | OTI - UNI **Programa:** Programa de Iniciación Tecnológica (PIT) 2026

Perfil del Alumno y Enfoque Pedagógico

Los estudiantes comprenden Docker Compose pero asumen que la comunicación entre contenedores requiere puertos abiertos locales. Hoy profundizarás en el funcionamiento de las Redes de Docker y el DNS interno. También abordarás la persistencia mediante volúmenes nombrados vs bind mounts y operaciones críticas: Healthchecks y backups (pg_dump).

Planificación de la Clase (3 Horas)

Bloque	Tiempo	Contenido
Redes y DNS	00:00 - 00:40	IPs dinámicas, resolución por alias, redes bridge/host/none
Volúmenes	00:40 - 01:20	Volatilidad, volumen nombrado vs bind mount
Seguridad	01:20 - 01:45	Aislamiento de puertos en bases de datos (redes independientes)
Receso	01:45 - 02:00	Break
Respaldo y Salud	02:00 - 02:40	Healthcheck, depends_on condition, backups pg_dump
Cierre	02:40 - 03:00	Taller práctico, tarea y cierre

Guión Paso a Paso del Docente

Introducción: Cómo comenzar la clase

Guión Sugerido (Lo que debes decir): "Buenas tardes. Hoy resolveremos dos problemas críticos en producción: cómo aislar la red de nuestros contenedores para resguardar la seguridad y cómo mantener la persistencia de nuestros datos. Aprenderemos a separar el tráfico de red, configurar volúmenes persistentes y programar respaldos automatizados."

Punto 1: Redes en Docker (Bridge, Host y None)

Guión Sugerido (Lo que debes decir): "Docker ofrece diferentes controladores de red: **bridge**: Red virtual bridge interna predeterminada. Si creamos una bridge personalizada, Docker habilita la resolución DNS interna por nombre del contenedor. **host**: Elimina el aislamiento de red entre contenedor y host, exponiendo puertos directamente. Solo en Linux. **none**: Aísla por completo el contenedor de cualquier red. Auditamos redes con **docker network ls** e inspeccionamos con **docker network inspect <red>**."

📄 Tabla de comunicación interna de la PPT:

Caso	Dirección correcta
Navegador → app	localhost:5000
App → PostgreSQL	db:5432
Otro servicio → app	web:5000

Regla práctica: Si el usuario final no necesita entrar directamente, NO publiques el puerto.

Punto 2: Aislamiento perimetral de base de datos

Guión Sugerido (Lo que debes decir): "En producción, no todas las aplicaciones deben verse entre sí. Nginx recibe solicitudes de internet y las pasa a la app web; esta consulta a la base de datos.

Crearemos una red 'frontend' para Nginx y la app, y otra red 'backend' para la app y Postgres. La base de datos queda totalmente invisible y segura, ya que Nginx NO tiene forma de comunicarse con ella. Además, retirando la directiva 'ports' de la BD en Compose, queda invisible desde internet."

📄 Arquitectura multi-red para proyectar:

```
Internet → [Nginx :80] ←frontend→ [Flask :5000] ←backend→ [Postgres :5432]
                                           (sin ports públicos)
```

Punto 3: Persistencia de datos: Volúmenes nombrados vs Bind Mounts

Guión Sugerido (Lo que debes decir): "Por definición, los contenedores son efímeros: si se eliminan, todo archivo en su capa de escritura temporal se borra. Para persistir datos usaremos:

- **Volúmenes Nombrados:** Gestionados por Docker en `/var/lib/docker/volumes/`. Ideal para bases de datos por su rendimiento y aislamiento. - **Bind Mounts:** Enlazan un directorio exacto del Host a una carpeta interna del contenedor. Indispensables en desarrollo para hot-reload del código."

📄 Tabla de la PPT:

Tipo	Uso	Ejemplo
Volumen nombrado	Datos que Docker gestiona	datos_postgres:/var/lib/postgresql/data
Bind mount	Montar carpeta del host	./app:/app (desarrollo)
tmpfs	Datos temporales en memoria	Solo Linux

📄 Ejemplo en Compose:

```
services:
  db:
```

```
image: postgres:16
volumes:
  - datos_postgres:/var/lib/postgresql/data

volumes:
  datos_postgres:
```

Punto 4: Healthchecks en Compose

Guión Sugerido (Lo que debes decir): "Un contenedor en ejecución NO siempre está listo para recibir peticiones. PostgreSQL tarda unos segundos en iniciar sus procesos.

Configuraremos un *healthcheck* en el servicio db usando *pg_isready*. En el servicio web Flask, declararemos que depende de la BD bajo la condición *service_healthy*, garantizando que Flask no inicie hasta que Postgres esté completamente listo, evitando caídas iniciales."

Ejemplo:

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
    interval: 5s
    timeout: 5s
    retries: 5

web:
  depends_on:
    db:
      condition: service_healthy
```

Punto 5: Respaldo y restauración (pg_dump y psql)

Guión Sugerido (Lo que debes decir): "Para respaldar una base de datos aislada sin exponerla, ejecutamos *pg_dump* internamente a través de Compose usando el flag *-T*. El flag *-T* desactiva la asignación de un pseudo-TTY, necesario para redirigir la salida a un archivo local de forma limpia.

Comando de backup: *docker compose exec -T db pg_dump -U appuser appdb > backups/appdb.sql*

Restauración: *docker compose exec -T db psql -U appuser appdb < backups/appdb.sql* "

Dinámica e Interacción en el Aula

- **Pregunta para lanzar al aula:** "¿Qué pasa con los datos de tu base de datos si haces *docker compose down* y no tienes volúmenes configurados?"

- *Respuesta:* Se pierden permanentemente. Sin volúmenes, la capa de escritura del contenedor es volátil.
- **Pregunta técnica:** "¿Flask puede hacer ping a PostgreSQL usando el nombre del servicio `db`?"
 - *Respuesta:* Sí, si están en la misma red bridge de Compose. Docker resuelve `db` a la IP interna del contenedor de PostgreSQL.

? FAQs

1. ¿docker compose down borra los volúmenes?

- No. `docker compose down` solo borra contenedores y redes. `docker compose down -v` borra TODO incluyendo volúmenes.

2. ¿Puedo acceder a PostgreSQL desde mi IDE local si no tiene ports?

- No directamente. Puedes usar `docker compose exec db psql -U usuario db_nombre` para acceder desde la CLI.

3. ¿Los bind mounts son seguros para producción?

- No se recomiendan en producción. Usar volúmenes nombrados para datos persistentes.

🔧 Práctica en Consola Paso a Paso

Lab 1: Preparar y levantar

```
cd codigo/sesion4
cp .env.example .env
docker compose up -d --build
docker compose ps
```

Lab 2: Probar la app

```
curl http://localhost:5000
curl http://localhost:5000/add
curl http://localhost:5000
```

Lab 3: Revisar red interna

```
docker network ls
docker network inspect sesion4_backend
docker compose exec web sh
# Dentro: ping db (debe resolver)
```

Lab 4: Verificar persistencia

```
curl http://localhost:5000/add      # Agregar datos
docker compose down                # Apagar SIN -v
docker compose up -d               # Volver a levantar
curl http://localhost:5000         # Los datos SIGUEN ahí
```

Lab 5: Backup de PostgreSQL

```
mkdir -p backups
docker compose exec -T db pg_dump -U appuser appdb > backups/appdb.sql
cat backups/appdb.sql
```

Lab 6: Restaurar backup

```
docker compose exec -T db psql -U appuser appdb < backups/appdb.sql
```

Trabajo para el Hogar

1. Levantar el stack de la sesión 4 con volúmenes persistentes.
2. Agregar datos con `curl http://localhost:5000/add` (mínimo 5 veces).
3. Ejecutar `docker compose down` y volver a levantar. Verificar que los datos persisten.
4. Crear un backup SQL con `pg_dump`.
5. **Entregable:** Captura de `docker compose ps`, `docker volume ls`, y el contenido del archivo de backup.

Gestión del Aula y Errores Comunes

Error	Solución
<code>connection refused</code> al inicio	PostgreSQL aún no está listo. Agregar healthcheck
<code>pg_dump: connection refused</code>	La BD aún está iniciando. Esperar o verificar con <code>docker compose ps</code>
<code>volume prune</code> borró datos	SIEMPRE verificar con <code>docker volume ls</code> antes de hacer prune
Red no resuelve nombres	Asegurarse de usar bridge personalizada, no la default

Evaluación — Test de la Sesión 4 (12 Preguntas)

1. **¿Qué sucede con los datos generados dentro de un contenedor si este es eliminado sin volúmenes configurados?**
 - a) Se guardan automáticamente en tu carpeta de usuario.

- b) Se pierden de forma permanente debido a la volatilidad del sistema de archivos del contenedor.
- c) Docker los almacena temporalmente en una caché oculta.
- d) Se suben de forma cifrada a Docker Hub.

2. ¿Cuál es la diferencia entre un Volumen Nombrado y un Bind Mount?

- a) Los volúmenes nombrados los gestiona Docker en una ruta reservada del host; los bind mounts enlazan una carpeta física exacta provista por el usuario.
- b) Los bind mounts borran los datos automáticamente al apagar.
- c) Los volúmenes nombrados solo funcionan en Linux nativo.
- d) Los bind mounts consumen el doble de RAM.

3. ¿En qué ruta de un Host Linux gestiona Docker los Volúmenes Nombrados?

- a) `/var/lib/docker/volumes/`
- b) `/etc/docker/storage/`
- c) `/tmp/docker/volumes/`
- d) `/home/user/.docker/volumes/`

4. ¿Qué comando te permite inspeccionar qué contenedores están conectados a una red y sus IPs?

- a) `docker network ls`
- b) `docker network inspect <nombre_red>`
- c) `docker network stats`
- d) `docker ip list`

5. ¿Por qué NO se debe publicar el puerto de PostgreSQL en producción?

- a) Aumenta la velocidad de consultas SQL.
- b) Evita exponer la base de datos a ataques externos, permitiendo conexión solo desde la red interna bridge.
- c) Permite que la BD corra sin contraseña.
- d) Libera memoria RAM a cero.

6. ¿Cómo se hace un respaldo SQL de PostgreSQL montado en Compose sin exponer el puerto?

- a) Deteniendo el contenedor y copiando `/var/lib/postgresql/data`.
- b) Ejecutando internamente: `docker compose exec -T db pg_dump -U usuario dbnombre > backup.sql`.
- c) Descargando los archivos SQL a través de un cliente web externo.
- d) Usando `docker cp` sobre los archivos temporales.

7. ¿Qué driver de red comparte la misma configuración de red, IPs y puertos del host (sin aislamiento)?

- a) Driver bridge
- b) Driver host
- c) Driver none/null
- d) Driver overlay

8. ¿Para qué sirve configurar un contenedor con el driver de red **none**?

- a) Para que consuma la mitad de ancho de banda.
- b) Para aislar el contenedor por completo de la red, desactivando conexiones para tareas seguras.
- c) Para permitir balanceo de carga automático.
- d) Para compartir archivos estáticos con Nginx.

9. ¿Qué tipo de montaje se declara con **--mount type=bind,src=/data,dst=/app,readonly**?

- a) Un volumen nombrado que se puede sobrescribir.
- b) Un enlace directo a una ruta local (Bind Mount) que el contenedor solo puede leer.
- c) Una red bridge de alta disponibilidad.
- d) Un contenedor con memoria temporal compartida.

10. ¿Qué comando elimina todos los volúmenes no asociados a contenedores activos?

- a) `docker volume rm --all`
- b) `docker volume prune`
- c) `docker system prune`
- d) `docker volume clean`

11. ¿Qué directiva de Compose garantiza que Flask no inicie hasta que PostgreSQL esté realmente listo?

- a) `depends_on: - db`
- b) `depends_on: db: condition: service_healthy` combinado con un `healthcheck` en el servicio db.
- c) `links: - db`
- d) `wait_for: db`

12. ¿Por qué se usa el flag **-T** en `docker compose exec -T db pg_dump`?

- a) Para ejecutar el comando en modo test sin escribir datos reales.
- b) Para desactivar la asignación de pseudo-TTY, permitiendo redirigir la salida limpiamente a un archivo.
- c) Para activar el modo Timeout y cancelar si demora mucho.
- d) Para forzar la conexión TCP en lugar de sockets UNIX.

 Solucionario Sesión 4

1-b | 2-a | 3-a | 4-b | 5-b | 6-b | 7-b | 8-b | 9-b | 10-b | 11-b | 12-b